RICE UNIVERSITY

The GPU Memory Hierarchy

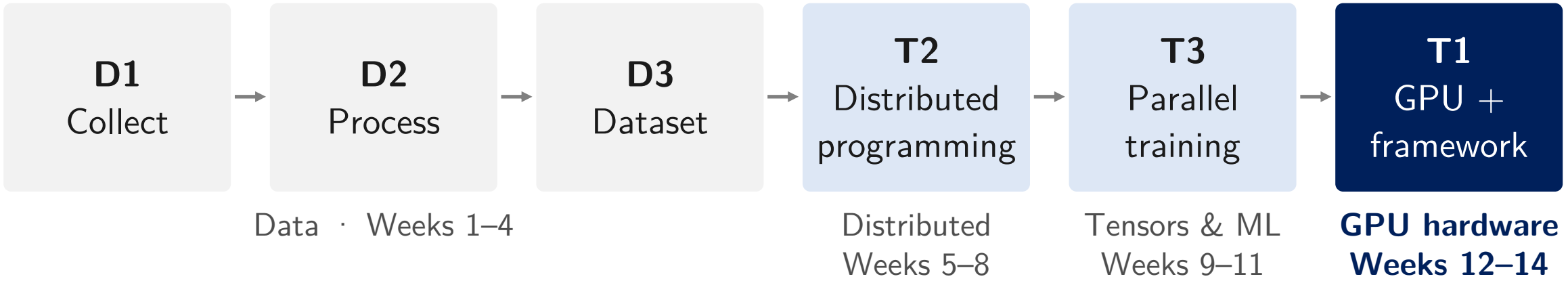
Registers, shared memory, L2, and HBM — and how a warp's access pattern decides how much bandwidth you get

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 13, Lecture 1

Where we are in the pipeline



In our pipeline

Today: **where data lives** on one GPU, and how to read it fast. **T1 checkpoint** this week; **capstone pairs** are set.

Week 12: warps and the execution model → today: memory → Lecture 2: the roofline

Why do we need a memory hierarchy?

One H100: 67 TFLOP/s of FP32 arithmetic
main memory (HBM) delivers **3.35 TB/s**

If every operand came from HBM, how much arithmetic could we use?

HBM: 3.35 TB/s ÷ 4 bytes
$\approx$ **0.84×10^{12} floats/s**

One FLOP per float loaded
$\approx$ **1.25%** of the peak

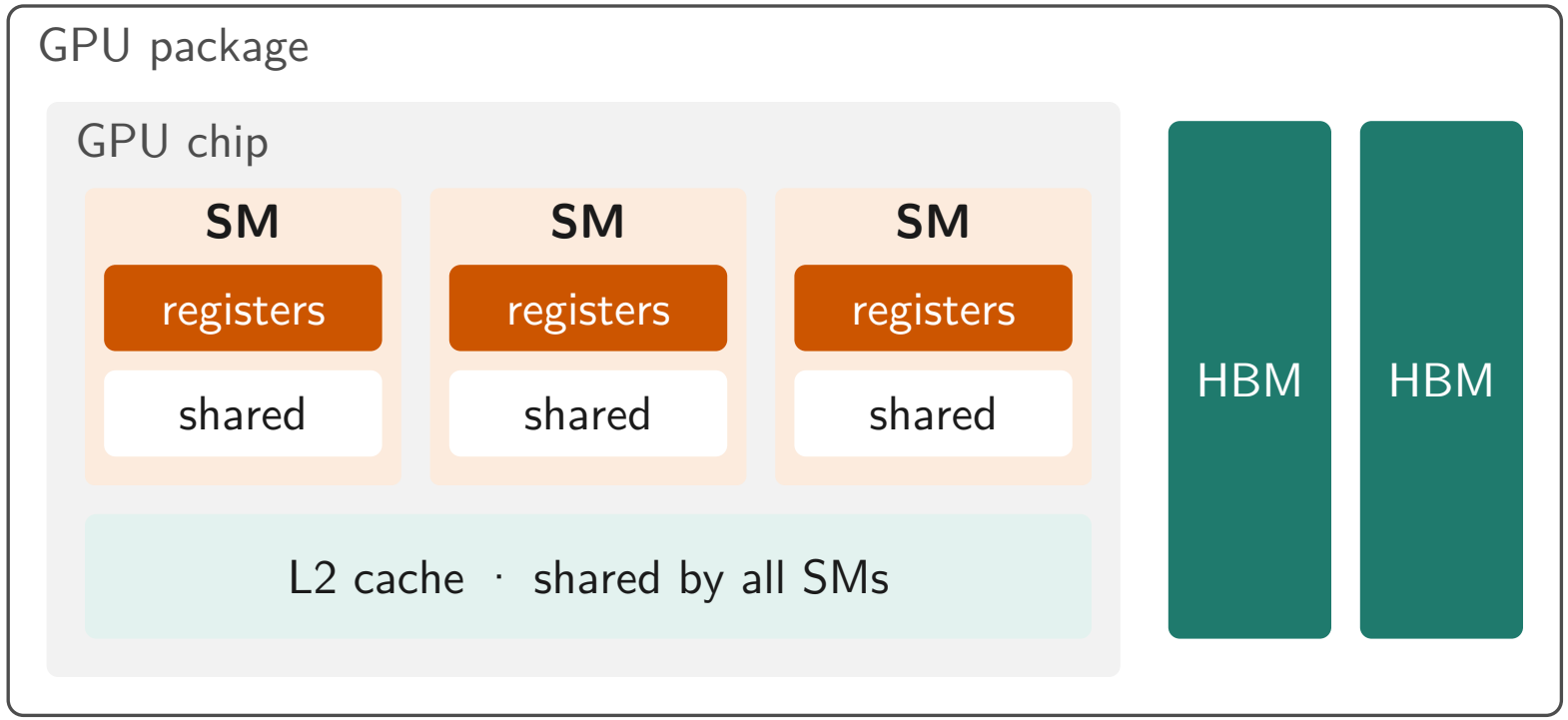
Key idea

Arithmetic is cheap; moving data is not. Every float from HBM must feed $\approx$ **80 FLOPs**, so reused data must come from somewhere **faster**.

Source: H100 datasheet (SXM5): FP32 peak and HBM bandwidth.

A memory hierarchy, in plain words

A **memory hierarchy**: small, fast memories next to the arithmetic; larger, slower ones further away. A **cache** keeps copies of recent data automatically.



Registers: one thread

Shared: one block

L2: the whole chip

HBM: tensors live here

Source: NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Where memory hierarchies show up

- **Every CPU** has one too: L1, L2, L3 caches in front of DRAM.
- **FlashAttention** (2022): attention computed in on-chip memory, never written to HBM.
- **Week 12, L2**: many warps hide the **latency**. Today: the **bytes**.
- **T1**: predict your kernel's memory traffic before you time it.

The price

Fast levels are **tiny**: 256 KB of registers per SM

Shared memory is managed **by hand** (or by Triton)

A bad **access pattern** wastes bandwidth

In our pipeline

Why a hierarchy? HBM cannot feed the arithmetic. Fast kernels **reuse** data from on-chip memory and ask HBM only for the bytes they need.

Source: Dao et al., "FlashAttention," NeurIPS 2022; H100 datasheet and whitepaper.

Learning goals for today

- Name the levels — **registers, shared memory, L2, HBM** — with capacity, latency, bandwidth.
- Explain **coalescing**, and count the bytes a warp moves versus the bytes it uses.
- Use **shared memory** to make a transpose coalesced, and avoid **bank conflicts**.
- Predict the **effective bandwidth** of an access pattern **before** measuring.

A puzzle to start

Kernel A — copy

y = x, 4096 × 4096 floats

reads 67 MB, writes 67 MB

Kernel B — transpose

y = `x.T`, same matrix

reads 67 MB, writes 67 MB

Same bytes in, same bytes out, no arithmetic. Does B run as fast as A?

Answer

No. One side of a naive transpose is **strided**; our count says up to **4.5×** slower. The **pattern** of access matters, not only the number of bytes.

Source: M. Harris, "An Efficient Matrix Transpose in CUDA C/C++," NVIDIA Developer Blog, 2013. Times derived from the H100 datasheet bandwidth.

Today's plan

1

Where data lives

How big, how fast, and who manages each level?

2

Coalescing

How do a warp's 32 loads turn into memory traffic?

3

Reuse from a faster level

If a value is needed many times, where should it come from?

Build

Predict effective bandwidth

Work in pairs. Count sectors first; measure in Lab 3.

Part 1

Where data lives

How big, how fast, and who manages each level?

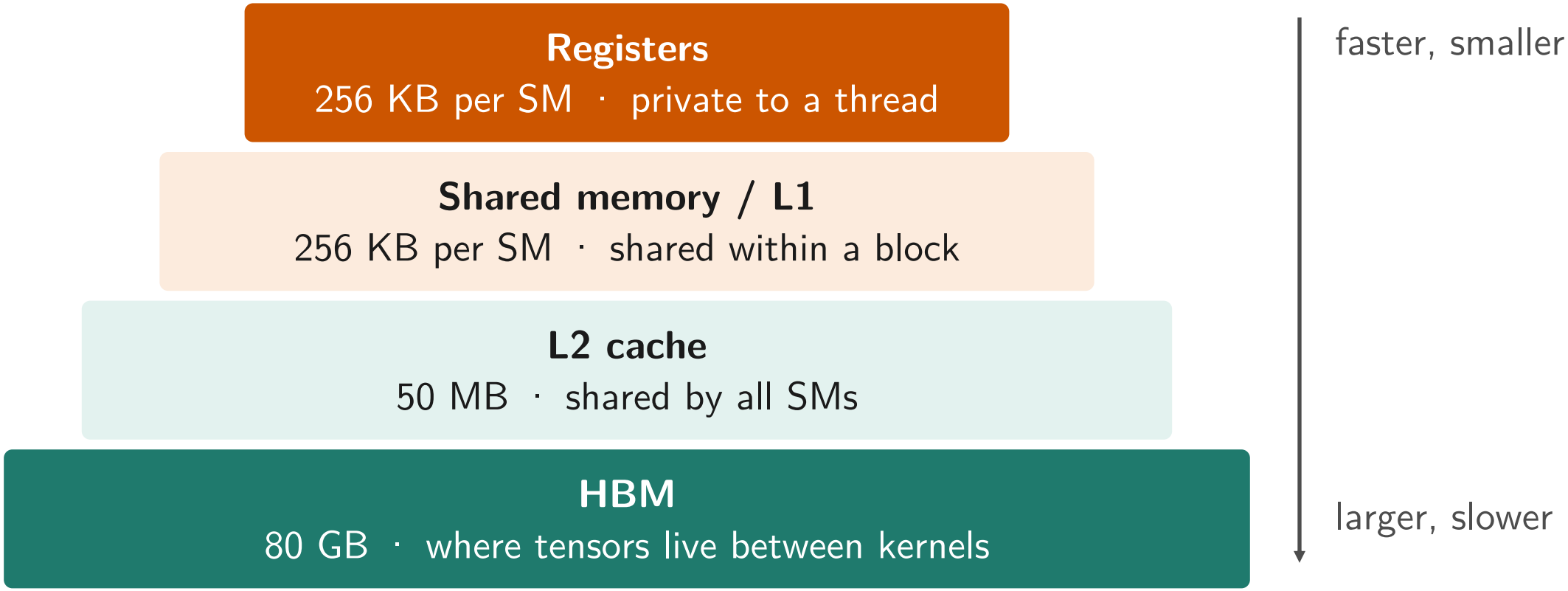
1 · Where data lives

2 · Coalescing

3 · Reuse from a faster level

Build · Predict effective bandwidth

The memory hierarchy



Key idea

Load from HBM **once**, reuse many times from registers or shared memory.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Registers: private and fastest

- Each thread's own variables: index, loaded values, partial sums.
- The **compiler** assigns them; you only choose how many values stay live.
- More per thread $\rightarrow$ **fewer warps** fit (Week 12, L2: occupancy).
- Too many $\rightarrow$ **spills** to "local memory", which lives in HBM.

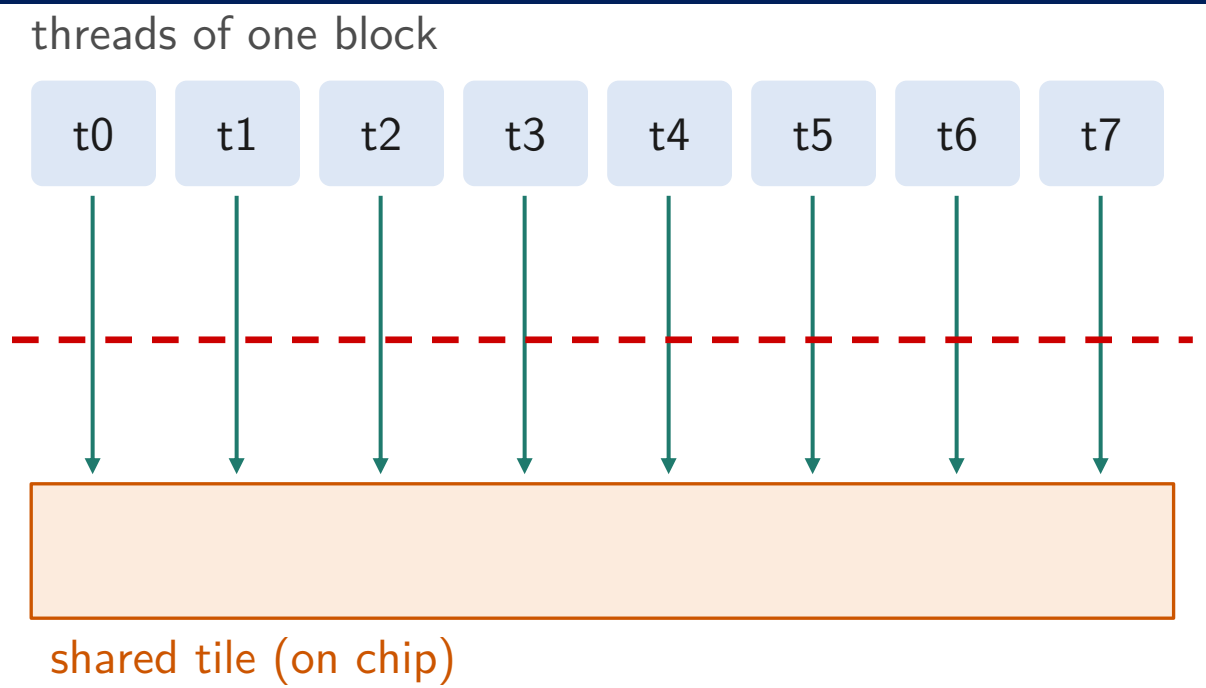
$$65{,}536 \text{ registers} \times 4 \text{ bytes}$$
$$= 256 \text{ KB per SM}$$
$$\times 132 \text{ SMs} \approx 33 \text{ MB}$$

Key idea

A value in a register is **free to reuse**. Keep reused values in registers; watch the count.

Source: NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100). H100 whitepaper.

Shared memory: a scratchpad for one block



1. each thread loads one value from HBM and stores it in the tile

2. barrier: wait until **every** thread has stored

3. any thread reads any value of the tile, **many times**, at on-chip speed

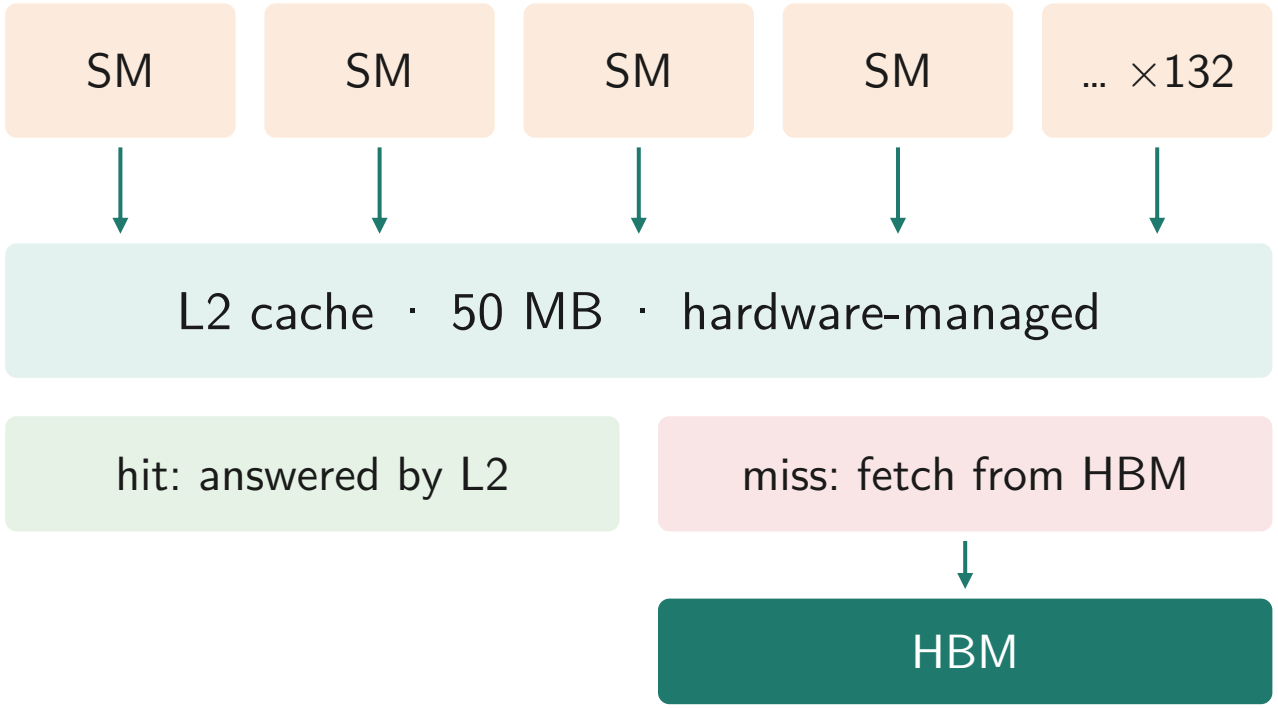
Up to **228 KB** per SM · one block only · CUDA: `__shared__`; Triton: automatic

Key idea

Shared memory is a **scratchpad**, not a cache: the kernel decides what goes in.

Source: NVIDIA CUDA C++ Programming Guide, "Shared Memory" (32 banks, 4 bytes wide, compute capability 9.0).

L2: the cache every SM shares



- Shared by **all** SMs: catches reuse **across blocks** and **across kernels**.
- Our MLP: 34,338 parameters $\approx$ **137 KB**, fits in L2 with room to spare.

Memory

L2 is automatic and **shared**: a bonus you get without asking, but cannot fully control.

Source: H100 whitepaper (50 MB L2); latencies are rounded, from microbenchmarks (e.g. Luo et al., IPDPS 2024).

HBM: large, fast, and far away

HBM
80 GB
3.35 TB/s
$\approx$ 500–800 cycles

- Holds **every tensor** between kernels.
- **High bandwidth**: many bytes per second.
- **Long latency**: each load waits hundreds of cycles.
- Week 12, L2: **many warps** hide the latency.

Watch out

Latency can be hidden; **bandwidth cannot**. Every byte asked for, used or not, costs bandwidth.

Source: H100 datasheet (SXM5); latencies are rounded, from microbenchmarks (e.g. Luo et al., IPDPS 2024).

The numbers to remember

Level	Capacity	Latency (approx.)	Bandwidth (approx.)
Registers	256 KB / SM	$\approx$ 1 cycle	—
Shared memory / L1	256 KB / SM	$\approx$ 30 cycles	$\approx$ 33 TB/s (all SMs)
L2 cache	50 MB	$\approx$ 200–300 cycles	several TB/s
HBM	80 GB	$\approx$ 500–800 cycles	3.35 TB/s

Key idea

Shared memory delivers $\approx$ **10×** HBM's bandwidth at a fraction of its latency: data reused **on chip** is almost free.

Source: H100 whitepaper and datasheet; latencies are rounded, from microbenchmarks (e.g. Luo et al., IPDPS 2024).

Check yourself

Two minutes with a neighbour.

- Who manages registers? Shared memory? L2?
- A kernel uses 128 registers per thread instead of 64. How many threads fit on an SM?
- Kernel 1 writes a 20 MB tensor; kernel 2 reads it right away. Where might the data come from?

Part 2

Coalescing

How do a warp's 32 loads turn into memory traffic?

1 · Where data lives

2 · Coalescing

3 · Reuse from a faster level

Build · Predict effective bandwidth

Memory moves in sectors

The unit of transfer is a **32-byte sector** (8 floats). A warp's load fetches every sector that holds at least one of its 32 addresses.

$$\text{efficiency} = \text{useful bytes} \;/\; \text{bytes moved}$$

$$\text{useful bytes per warp} = 32 \times 4 = 128$$

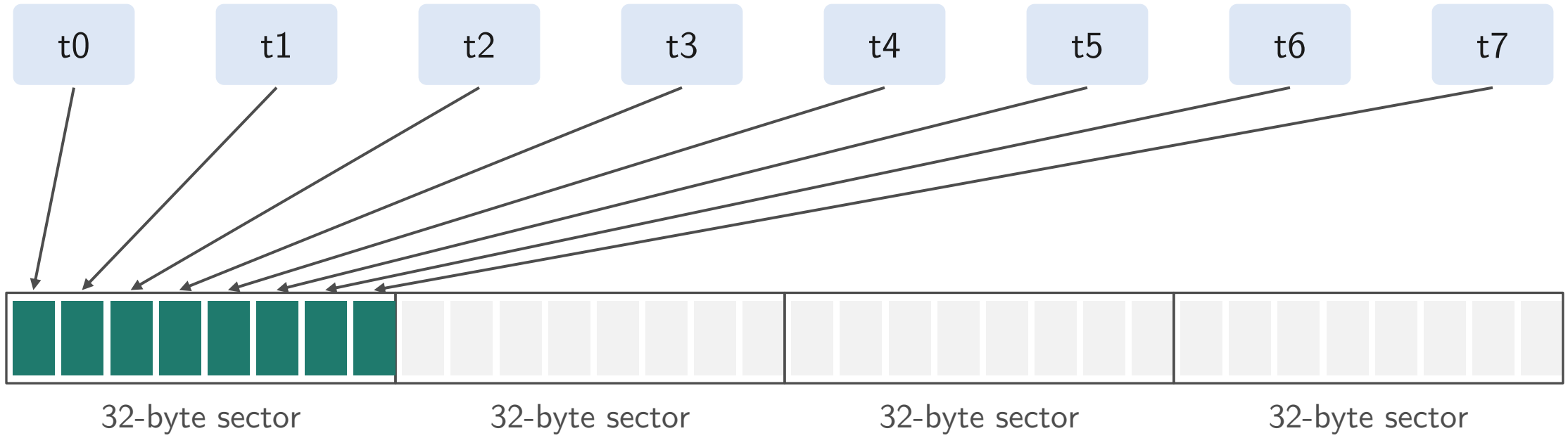
$$\text{effective bandwidth} = \beta \times \text{efficiency}$$

Memory

Coalesced: the warp's addresses fall into as few sectors as possible. Best case: 4 sectors for 128 bytes.

Source: NVIDIA CUDA C++ Best Practices Guide, "Coalesced Access to Global Memory" (32-byte sectors).

Coalescing: neighbours read neighbours



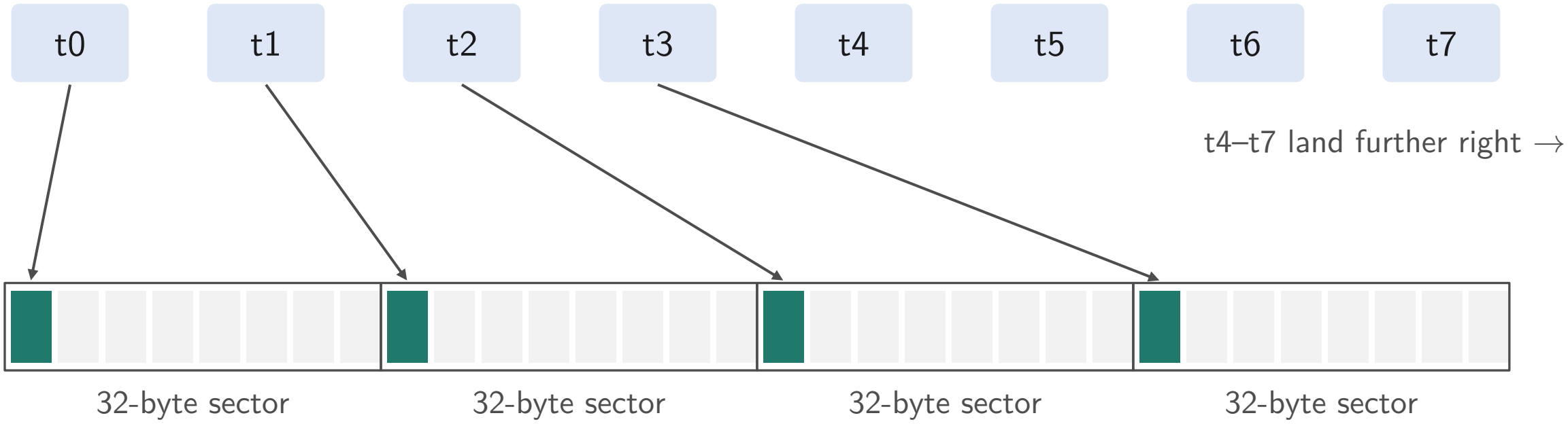
top: 8 of a warp's 32 threads · bottom: memory, one square per float

Memory

8 threads → one full sector; a warp of 32 → **128 bytes, all used**. Efficiency 100%.

Source: NVIDIA CUDA C++ Best Practices Guide, "Coalesced Access to Global Memory".

Strided access wastes bandwidth



top: 8 of a warp's 32 threads · bottom: memory, one square per float

Watch out

Each thread pulls a whole 32-byte sector for 4 useful bytes: 1,024 bytes moved, 128 used → **12.5%**.

Source: NVIDIA CUDA C++ Best Practices Guide, "Strided Accesses".

Efficiency falls with the stride

Stride s (floats)	Sectors per warp	Efficiency
1	4	100%
2	8	50%
4	16	25%
≥ 8	32	12.5%

$$s \leq 8: \ \text{sectors} = 4s$$

$$\text{efficiency} = 1 \ / \ s$$

$$s \geq 8: \ \text{efficiency} = 1/8$$

Watch out

Past stride 8, every thread costs a whole sector: **12.5%**, an 8× loss — 3.35 TB/s becomes ≈ **0.42 TB/s**.

Source: NVIDIA CUDA C++ Best Practices Guide, "Coalesced Access to Global Memory" (32-byte sectors).

Row-major layout and strides

0,0	0,1	0,2	0,3	0,4	0,5
1,0	1,1	1,2	1,3	1,4	1,5
2,0	2,1	2,2	2,3	2,4	2,5
3,0	3,1	3,2	3,3	3,4	3,5

A: 4×6, row 1 highlighted

$$\text{offset}(i, j) = i \cdot N + j$$

$$\text{strides} = (N, 1)$$



the same 24 numbers in memory: row 1 is contiguous

Key idea

A **row** has stride 1 (coalesced); a **column** has stride N (strided). `A.T` swaps strides, not data.

Your indexing decides coalescing

```
x = torch.randn(4096, 4096, device="cuda")
x.stride()           # (4096, 1)
xt = x.T             # a view: no data moved
xt.stride()          # (1, 4096)
xt.is_contiguous()   # False
y = xt.contiguous() # runs a transpose kernel
```

- **CUDA**: `threadIdx.x` → last dim
- **Triton**: `tl.arange` → last dim
- **PyTorch**: check `stride()`

Key idea

Neighbouring threads → neighbouring addresses. A transposed **view** is free; walking it is not.

Source: PyTorch documentation: torch.Tensor.stride, Tensor.contiguous.

Check yourself

Two minutes with a neighbour.

- A warp reads 32 consecutive **BF16** values (2 bytes each). Sectors? Efficiency?
- A warp reads every 4th float. Efficiency, and effective bandwidth on an H100?
- Why is `x.T` free while `x.T.contiguous()` is not?

Part 3

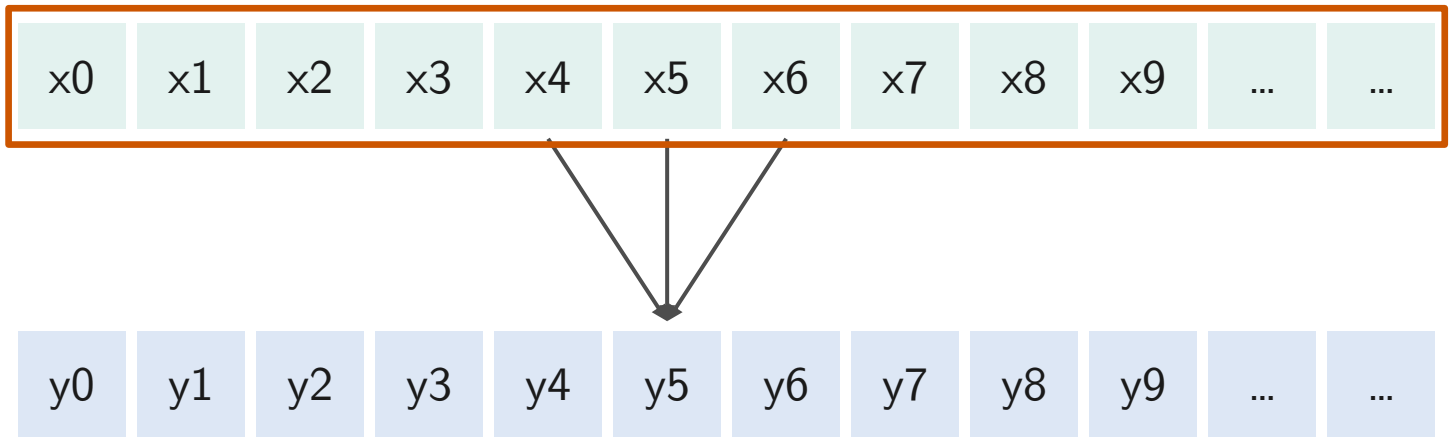
Reuse from a faster level

If a value is needed many times, where should it come from?

1 · Where data lives | 2 · Coalescing | **3** · Reuse from a faster level | **Build** · Predict effective bandwidth

Reuse: load once, use many times

block: load B + 2 inputs into shared memory **once**



each output reads 3 inputs; each input is read 3 times

naive: 3 loads per output

shared: $(B + 2) \;/\; B$

$= 258 \;/\; 256 \approx 1.01$

Key idea

A value used **r** times costs **one** slow load and r − 1 fast ones. (Week 14, L1: the same idea for matrix multiply.)

Transposing: one side is always strided



read a row: stride 1, coalesced

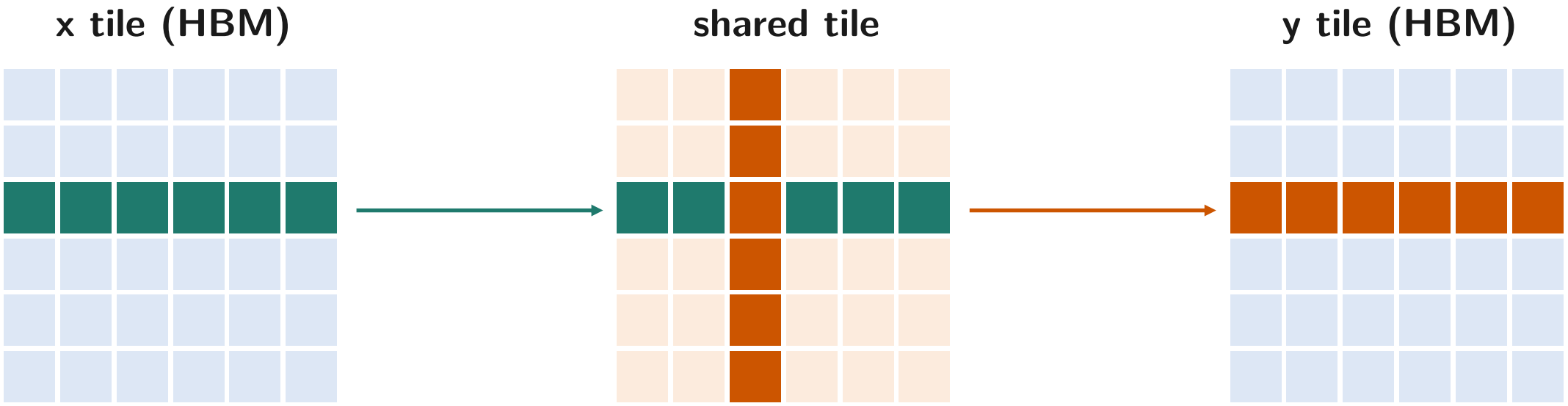
write a column: stride N, 12.5%

Watch out

A transpose reads along one dimension and writes along the other: in HBM, **one side is strided**.

Source: M. Harris, "An Efficient Matrix Transpose in CUDA C/C++," NVIDIA Developer Blog, 2013.

Transpose through shared memory

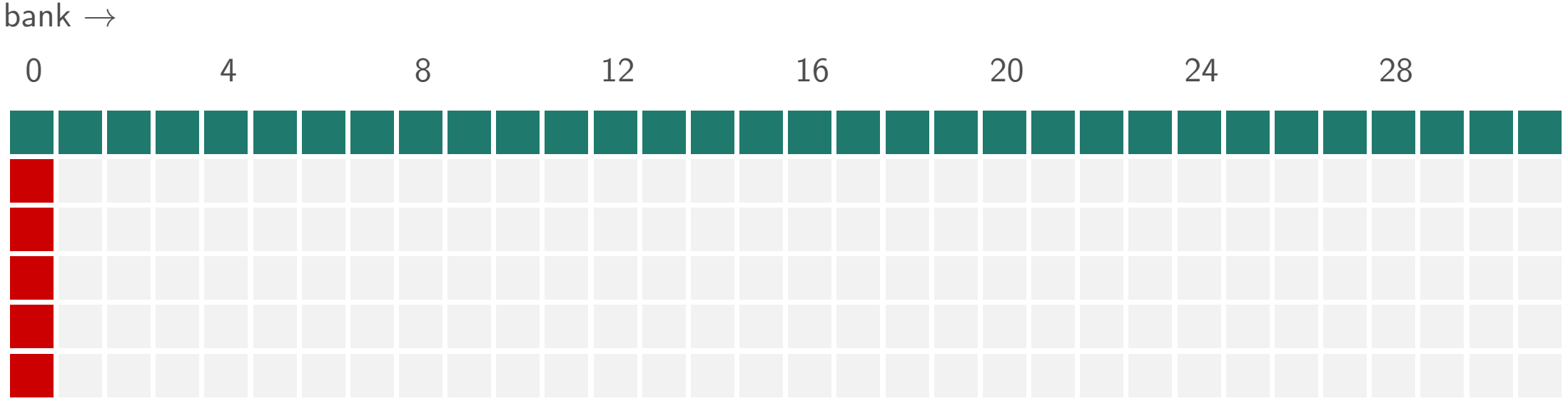


Key idea

Both HBM accesses are now **coalesced**; the awkward, strided step happens **on chip**.

Source: M. Harris, "An Efficient Matrix Transpose in CUDA C/C++," NVIDIA Developer Blog, 2013.

Shared memory has 32 banks



a row: 32 different banks → **1 cycle**

a column: all in bank 0 → **32 cycles**

Watch out

Word w lives in bank **w mod 32**. Threads of a warp that hit **different words of one bank** wait in line: a **bank conflict**.

Source: NVIDIA CUDA C++ Programming Guide, "Shared Memory" (32 banks, 4 bytes wide, compute capability 9.0).

Padding removes the conflict

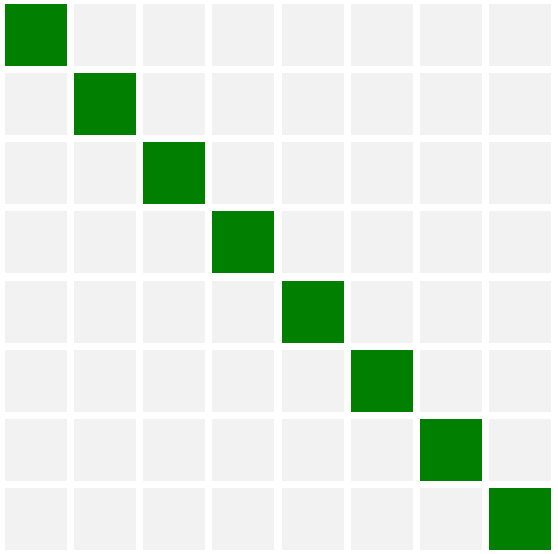
$$\text{tile}[32][32]:\ \ \text{bank} = (32\ r + c) \bmod 32 = c$$

$$\text{column } c = 0:\ \ \text{all 32 threads} \to \text{bank } 0$$

$$\text{tile}[32][33]:\ \ \text{bank} = (r + c) \bmod 32$$

$$\text{column } c = 0:\ \ \text{thread } r \to \text{bank } r$$

banks 0–7, rows 0–7



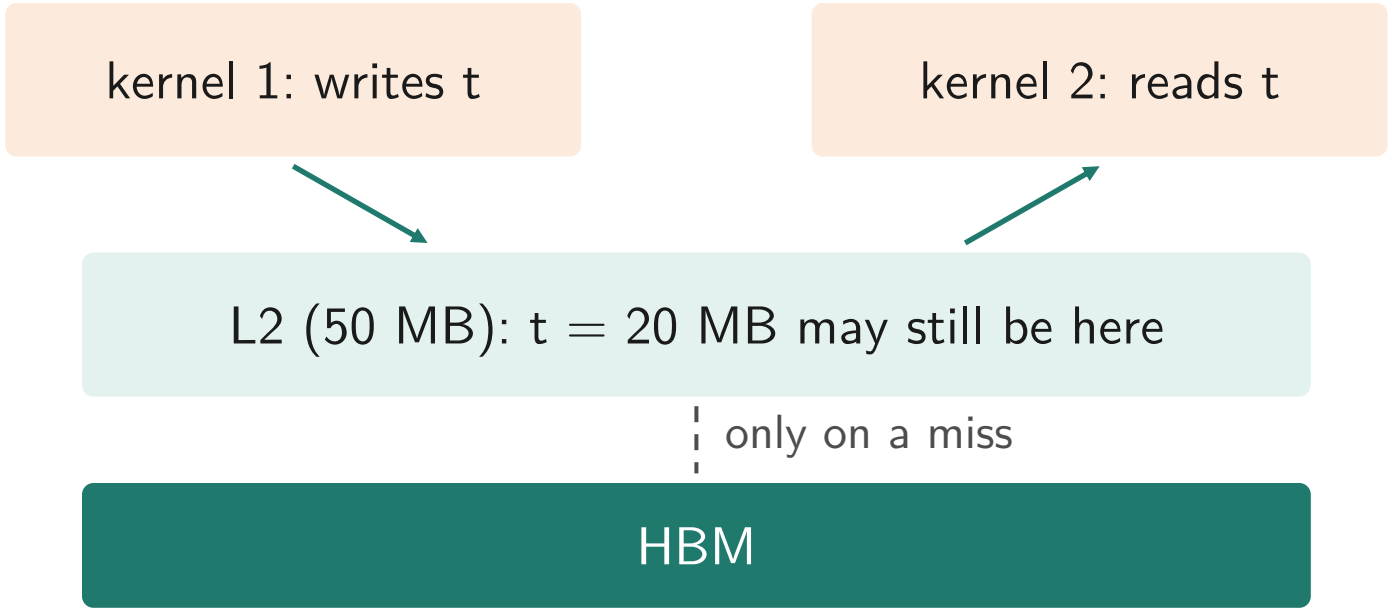
padded column: one per bank

Fix

One unused column (**3%** more shared memory) removes a **32×** slowdown.

Source: NVIDIA CUDA C++ Programming Guide, "Shared Memory" (32 banks, 4 bytes wide, compute capability 9.0).

L2 catches reuse across kernels



- Small tensors can **beat** the HBM estimate.
- Benchmarks: **flush L2** or use inputs $\gg$ 50 MB.

Memory

L2 is a bonus you **cannot control**; shared memory and registers are the levels you can.

Source: H100 whitepaper (50 MB L2); triton.testing.do_bench (flushes L2 before each run).

Check yourself

Two minutes with a neighbour.

- A 5-point blur, blocks of 256 outputs: loads per output, naive versus shared memory?
- In the transpose, why is a strided pattern fine in shared memory but not in HBM?
- A warp reads column 0 of `float tile[32][64]`. Conflict? And with `[32][65]`?

IN-CLASS BUILD

Predict effective bandwidth

Work in pairs. Count sectors first; measure in Lab 3.

1 · Where data lives | 2 · Coalescing | 3 · Reuse from a faster level | **Build** · Predict effective bandwidth

Exercise: four access patterns

Pattern (one warp, FP32)	Sectors	Efficiency	Useful TB/s
1. `x[i]`, consecutive i			
2. `x[2 * i]`, every other float			
3. `p[i].x` in an array of {x, y, z}			
4. a column of a 4096×4096 matrix			

USE

32-byte sectors · 4-byte floats

$\beta = 3.35$ TB/s; useful $= \beta \times$ efficiency

STRETCH

(a) `x[i + 1]`: shifted by one float.

(b) Time for 10^8 useful floats: 4 vs 1?

Source: NVIDIA CUDA C++ Best Practices Guide, "Coalesced Access to Global Memory" (32-byte sectors).

Exercise: solutions

Pattern (one warp, FP32)	Sectors	Efficiency	Useful TB/s
1. `x[i]`, consecutive i	4	100%	3.35
2. `x[2 * i]`, every other float	8	50%	1.68
3. `p[i].x` in an array of {x, y, z}	12	33.3%	1.12
4. a column of a 4096 × 4096 matrix	32	12.5%	0.42

Stretch: (a) 5 sectors, **80%**. (b) **0.96 ms** versus **0.12 ms**: 8× longer.

Key idea

Count sectors, not elements. **Structure of arrays** beats array of structures when a kernel reads one field.

Source: NVIDIA CUDA C++ Best Practices Guide, "Coalesced Access to Global Memory" (32-byte sectors).

How to measure it in Lab 3

```
from triton.testing import do_bench
x = torch.randn(2**28, device="cuda")  # 1 GiB
for s in (1, 2, 3, 8, 32):
    v = x[::s]                         # a strided view
    ms = do_bench(lambda: v.sum(),
                  return_mode="median")
    gbs = 4 * v.numel() / ms / 1e6
    print(s, round(gbs), "GB/s useful")
```

- Timing as in **Week 12, L2**: warm up, synchronize, median.
- Input $\gg$ **L2**, so HBM is what you measure.
- Compare the **ratios** with your table.

In our pipeline

T1 checkpoint: next to every kernel, a **predicted** traffic and a **measured** GB/s.

Source: triton.testing.do_bench (Triton source); PyTorch documentation (slicing returns a view).

Key takeaways

- Registers $\rightarrow$ shared memory $\rightarrow$ L2 $\rightarrow$ HBM: each level down is **larger and slower**.
- HBM cannot feed the arithmetic: every float must feed $\approx$ **80 FLOPs**, so **reuse** on chip.
- Memory moves in **32-byte sectors**; a **coalesced** warp uses all of them.
- Stride s wastes bandwidth: efficiency $\mathbf{1/s}$, down to $\mathbf{12.5\%}$.
- **Shared memory** turns a strided transpose coalesced; **pad** to avoid bank conflicts.
- **L2** catches reuse across kernels — a bonus, not a plan.

Next time, and readings

When	What
Lecture 2 (this week)	Arithmetic intensity and the roofline model
This week	T1 checkpoint; capstone pairs set
Week 14, Lecture 1	Precision and the anatomy of a matrix multiply; then Lab 3 due

READ BEFORE LECTURE 2

CUDA C++ Best Practices Guide: "Coalesced Access to Global Memory" and "Shared Memory".

Williams, Waterman & Patterson, "Roofline," CACM 2009 (for Lecture 2).